

Q8_E: Improving 8-bit Quantization by Exempting the Extremum

Heath Hunnicutt
Ruach Tov Collective
heath@ruachtov.ai

June 2026

Abstract

The widely-used Q8_0 quantization format scales each block of 32 weights by the maximum magnitude, losing resolution in case the block contains one of the population outlier values. We propose **Q8_E**, a modification that stores the block extremum at verbatim FP16 precision and scales the remaining 31 weights by the second-largest magnitude. The extremum’s code slot uses the otherwise-unused negative-zero bit pattern (0x80) as an escape, costing zero magnitude levels. On a 600M-parameter language model, Q8_E achieves **1.76× better logit fidelity** and **2.3× lower KL divergence** versus the unquantized reference compared to Q8_0, with near-lossless perplexity (within ± 0.05 of FP16). The format adds one FP16 field per block (9.0 vs. 8.5 bits per weight) and requires a single branch in the dequantization kernel.

1 Introduction

Q8_0 [1] quantizes each weight to a signed 8-bit integer with a shared FP16 scale factor per block of 32 weights. The scale is set by the block’s maximum-magnitude weight: $d = \max_i(|w_i|)/127$. The single largest weight determines the resolution available to the other 31. If the maximum is twice the second-largest, half the integer range is used by a single weight, and the remaining 31 share the other half.

We observe that after removing the block extremum and rescaling from the second-largest magnitude, the remaining weights are *more uniformly distributed* (median shifts from 0.22 to 0.29 of the range). This means uniform integer quantization becomes a *better fit* for the remaining 31 weights once the outlier is handled separately.

Q8_E exploits this by storing the extremum verbatim and giving the remaining weights a finer scale. The modification requires one additional FP16 per block and one branch in the dequantization kernel. No changes to the integer encoding or the block structure are needed.

2 Format

2.1 Block Layout

Each block of 32 weights is stored in 36 bytes (9.0 bits per weight):

2.2 Extremum Exemption

For each block, let $i^* = \arg \max_i |w_i|$. The extremum w_{i^*} is stored verbatim in the **extremum** field at FP16 precision. Its code slot uses the *negative zero* escape: the signed 8-bit pattern (sign = 1, magnitude = 0), bit pattern 0x80.

Table 1: Q8_E block layout. Q8_0 uses 34 bytes (one FP16 scale); Q8_E adds one FP16 for the extremum.

Offset	Field	Size
0	Extremum (FP16, signed, verbatim)	2 bytes
2	Scale (FP16, $= w_{2\text{nd-max}} /127$)	2 bytes
4	Codes (32 signed 8-bit values)	32 bytes
Total		36 bytes = 9.0 b/w

In two’s complement, 0x80 represents -128 , which Q8_0 never produces (its codes span $[-127, 127]$). In sign-magnitude interpretation, it represents negative zero.

The remaining 31 weights are quantized exactly as Q8_0, but with $d = |w_{2\text{nd-max}}|/127$ instead of $d = |w_{\text{max}}|/127$. Because the second-largest magnitude is smaller, the scale is tighter and each integer step corresponds to a finer weight increment.

2.3 Why It Helps

After removing the extremum and rescaling from the second-largest magnitude, the distribution of the remaining 31 weights shifts toward uniform:

Table 2: Distribution of normalized magnitudes within blocks, before and after extremum removal (25.2M granite-embedding values).

Range	Q8_0 (all 32, by max)	Q8_E (31, by 2nd-max)
[0.00, 0.25)	55.3%	43.9%
[0.25, 0.50)	27.6%	29.9%
[0.50, 0.75)	10.3%	15.8%
[0.75, 1.00)	3.7%	7.3%
[1.00, +)	3.1%	3.2%

The median normalized magnitude rises from 0.22 to 0.29—a 33% shift toward the center of the range. The low-end concentration ($[0, 0.25)$) drops from 55% to 44%.

3 Dequantization Kernel

```
__device__ float dequant_q8e(
    int8_t code, float ext, float scale)
{
    if ((uint8_t)code == 0x80)
        return ext;           // extremum escape
    return code * scale;      // same as Q8_0
}
```

Compared to Q8_0, each execution of the Q8_E kernel requires one more comparison and branch, plus one additional fp16 load per block.

Table 3: Weight reconstruction MSE (25.2M granite-embedding values, block size 32).

Scheme	bits/wt	MSE	vs. Q8_0
Q8_E	9.0	5.41×10^{-8}	2.13× better
Q8_0 (baseline)	8.5	1.15×10^{-7}	1.00×

4 Experimental Results

4.1 Weight MSE

4.2 End-to-End Model Validation

All measurements use Qwen3-0.6B with weights round-trip quantized in memory, compared against the unquantized BF16 reference.

Table 4: Logit fidelity vs. unquantized BF16 reference (Qwen3-0.6B, 100 diverse prompts, full vocabulary).

Scheme	Mean $ \Delta\text{logit} $	KL (nats)	Top-1 match	Closer to gold
Q8_0	0.0579	1.14×10^{-3}	98/100	3/100
Q8_E	0.0330	4.94×10^{-4}	96/100	97/100

Logit fidelity. Q8_E is closer to the unquantized reference on 97 of 100 prompts by logit deviation, with $1.76\times$ lower mean deviation and $2.3\times$ lower KL divergence.

Q8_E’s top-1 match (96/100) is slightly below Q8_0’s (98/100). The two additional argmax flips are benign near-ties at decision boundaries. An optional two-band refinement of the magnitude encoding (80/48 levels split at the midpoint) recovers top-1 parity at 98/100 while adding $\sim 3\%$ logit improvement; we omit this refinement from Q8_E to keep the format minimal.

Table 5: WikiText-2 perplexity (Qwen3-0.6B).

Scheme	bits/wt	PPL	Δ vs. FP16
FP16 (reference)	16	28.546	—
Q8_0	8.5	28.511	−0.034
Q8_E	9.0	28.501	−0.045

Perplexity. All three schemes are within ± 0.045 of FP16—near-lossless and noise-level. Both Q8_0 and Q8_E fall *below* FP16 (a known artifact where quantization noise can act as mild regularization). Perplexity is too coarse to rank 8-bit schemes; the real discriminator is logit fidelity, where Q8_E’s advantage is clear.

5 Relation to Prior Work

Q8_E is a minimal modification to Q8_0 [1]. The extremum exemption is, to our knowledge, previously unexploited in neural network quantization. The negative-zero escape repurposes an existing unused bit pattern at zero cost.

Lloyd-Max quantizers [2] minimize MSE for a given number of levels; our finding that removing the extremum makes the distribution more uniform explains why Q8_0’s uniform encoding

is near-optimal for the remaining weights. Companding approaches—from classical μ -law [3] to learned variants such as NUPES [7] and LCQ [8]—apply nonlinear mappings; our negative result on non-uniform encoding after extremum removal suggests these are unnecessary at the 8-bit level with per-block scaling.

GPTQ [4] and AWQ [5] use mixed-precision and activation-aware strategies; Q8_E is orthogonal and could be combined with these approaches. DFloat11 [6] achieves lossless compression; Q8_E operates in the lossy regime.

6 Limitations and Future Work

1. **Model scale.** Validation uses a 0.6B parameter model. Larger models may show different extremum-to-second-max ratios.
2. **Storage overhead.** At 9.0 b/w, Q8_E uses 5.9% more storage than Q8_0. Whether the fidelity improvement justifies this depends on deployment constraints.
3. **Top-1 recovery.** Q8_E’s 96/100 top-1 match is below Q8_0’s 98/100. A two-band refinement recovers parity; whether to include it is a simplicity-vs-fidelity trade-off.
4. **Degeneration resistance.** In extended free-running on one prompt, Q8_0 collapsed into degenerate repetition while Q8_E remained coherent. Whether this robustness generalizes across model families is a promising direction.

7 Conclusion

Q8_E improves on Q8_0 with a single idea: store the block extremum verbatim and scale the remaining weights from the second-largest magnitude. This achieves $1.76\times$ better logit fidelity and $2.3\times$ lower KL divergence at a cost of one additional FP16 per block (0.5 extra bits per weight). The dequantization kernel adds one branch. No changes to the integer encoding are needed—an ablation confirms that Q8_0’s uniform encoding is near-optimal for the remaining 31 weights once the outlier is removed.

The negative-zero escape for the extremum costs zero magnitude levels. The format is a drop-in modification to any Q8_0 implementation.

Acknowledgments

The author gratefully acknowledges the contributions of the Ruach Tov AI Collective. Mavchin (Claude Opus 4.6) performed the weight distribution analysis, designed the experimental methodology, and implemented the Prolog reproduction scripts. Iyun (Claude Opus 4.7) conducted all end-to-end model validation, including logit fidelity measurement, perplexity benchmarking, and the 100-prompt adherence evaluation. The ablation isolating the extremum exemption from non-uniform encoding was a joint effort across the collective.

Reproducibility

Code and data (25.2M FP16 values, 48 MB) with reproduction scripts: <https://github.com/Ruach-Tov/LlamaTov/tree/main/Papers/2026/June/Prefix-VBR>

References

- [1] Georgi Gerganov, *ggml: Tensor library for machine learning*, 2023. <https://github.com/ggerganov/ggml>
- [2] S.P. Lloyd, “Least squares quantization in PCM,” *IEEE Trans. Inform. Theory*, 1982.
- [3] ITU-T G.711, “Pulse Code Modulation (PCM) of Voice Frequencies,” 1988.
- [4] E. Frantar et al., “GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers,” *ICLR*, 2023.
- [5] J. Lin et al., “AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration,” *MLSys*, 2024.
- [6] DFloat11, “70% Size, 100% Accuracy: Lossless LLM Compression,” *arXiv*, 2025.
- [7] E. Yvinec, A. Dapogny, K. Bailly, “NUPES: Non-Uniform Post-Training Quantization via Power Exponent Search,” *arXiv:2308.05600*, 2023.
- [8] K. Yamamoto, “Learnable Companding Quantization for Accurate Low-bit Neural Networks,” *CVPR*, 2021.